

Graph learning notes

Round 1

1. Graph Basics (图基础)

图由：

- 点 (nodes / vertices)
- 边 (edges)

组成。

图可以：

- 有向 / 无向
- 有权 / 无权

一些核心概念

Path (路径)

一串连续边。

Cycle (环)

起点终点相同的路径。

有环 (cyclic) 无环 (acyclic)

Connected (连通)

任意两点之间都能到达。

Component (连通块)

最大的连通子图。

2. Tree (树)

树 = 连通 + 无环

A connected acyclic graph is a **tree**.

树的重要性质

n 个点的树： $|E|=|V|-1$

即 n-1 条边。

树删一条边 → 图断开

树加一条边 → 出现环

3. DFS (Depth-First Search)

| 一路往深处走，走不动再回退。

DFS 的典型用途

1. 遍历图
2. 检测环
3. 找连通块
4. Topological Sort

DFS 的数据结构

栈（递归本身也是栈）

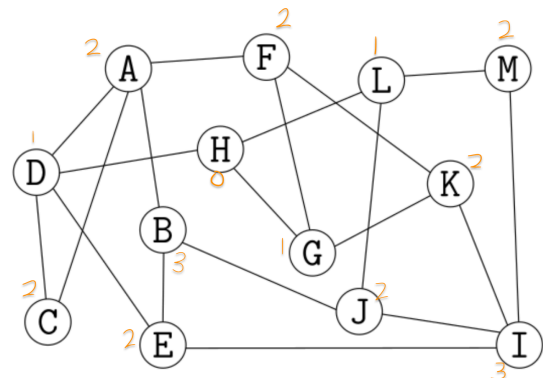
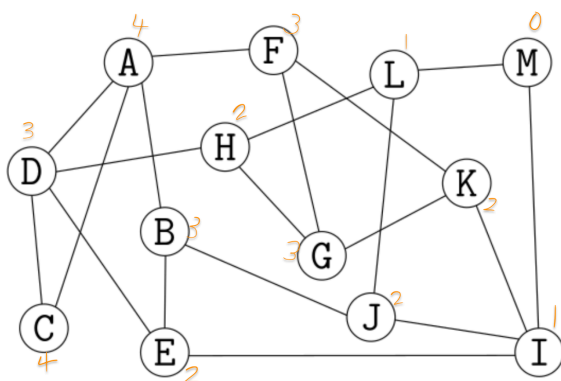
4. BFS (Breadth-First Search)

| 一层一层扩展。

BFS 核心

layer (层)

- 距离起点 1 条边
- 距离起点 2 条边
- ...



BFS 最重要性质

| BFS finds shortest unweighted paths

BFS 求无权图最短路

queue

DFS vs BFS

```
DFS(s)
stack = empty list of (node, neighbor index) pairs
add pair (s, 0) to the end of stack
mark s as visited
while stack is not empty:
    read a current pair (x, i) from the end of stack
    while x.neighbors[i] exists and is a visited node:
        i = i + 1
    if x.neighbors[i] exists:
        update the current pair in stack to (x, i+1)
        mark the node x.neighbors[i] as visited
        add the pair (x.neighbors[i], 0) to the end of stack
    else:
        remove the current pair from the end of stack
```

```
BFS(s)
layers = empty list of layer node lists
newLayer = empty node list
add start node s to newLayer
mark s as visited
while newLayer is not empty:
    add newLayer to the end of layers
    expandLayer = empty node list
    for each node x in newLayer:
        for each neighbor node y of x:
            if y has not been visited:
                mark y as visited
                add y to expandLayer
    newLayer = expandLayer
return layers
```

DFS	BFS
深入到底	一层层
stack	queue
适合环/拓扑	适合最短路
recursive 常见	queue 常见

5. Spanning Tree (生成树)

| 能连接所有点的一棵树。

例如：

- DFS tree
- BFS tree

6. Minimum Spanning Tree (MST)

| 在所有 spanning tree 中，总边权最小。

MST 的本质

✅ 整体连通代价最小

7. Graph Cut (割)

Cut, Cutset 的意思

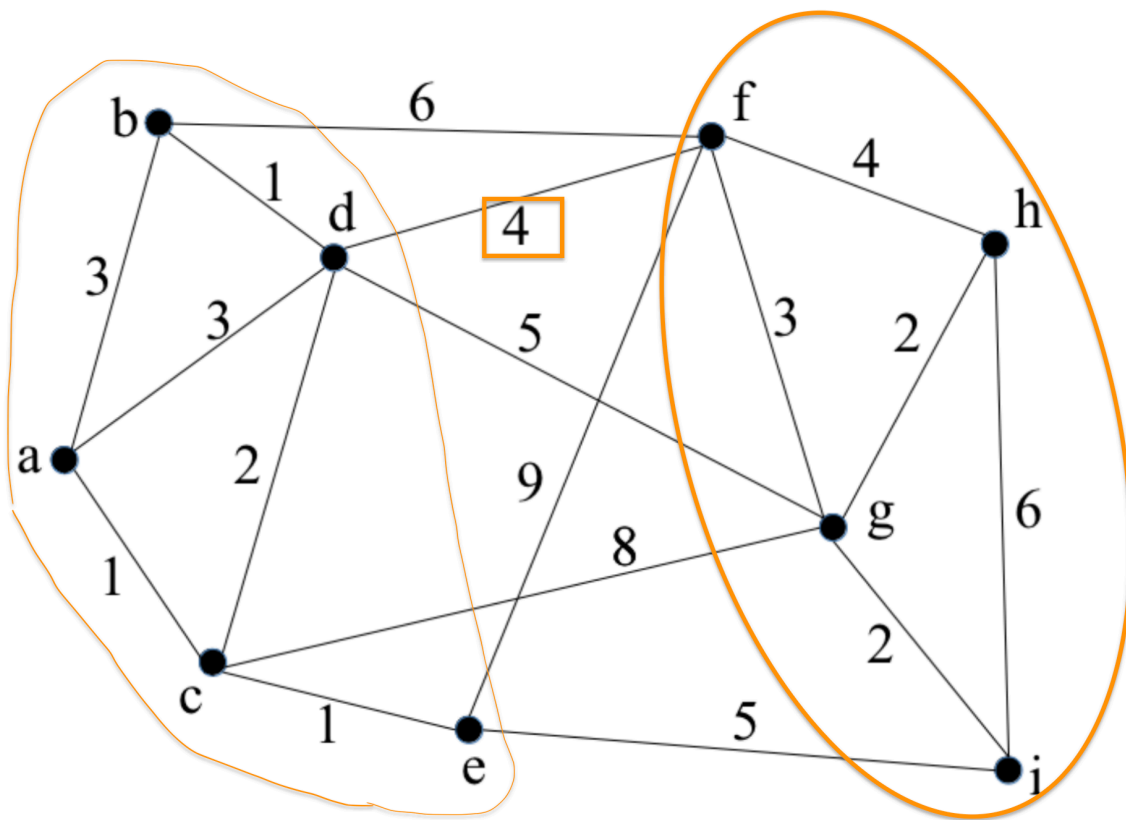
把图分成两部分。

连接两部分的边：叫 cutset。

MST 最重要性质

| cutset 中最小边一定可以属于 MST

这其实就是：Kruskal / Prim 正确性的来源。



8. Prim's Algorithm (Prim 最小生成树)

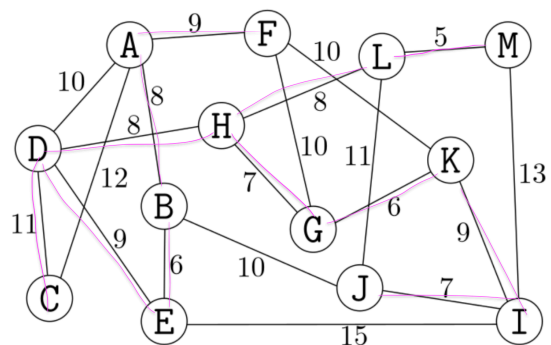
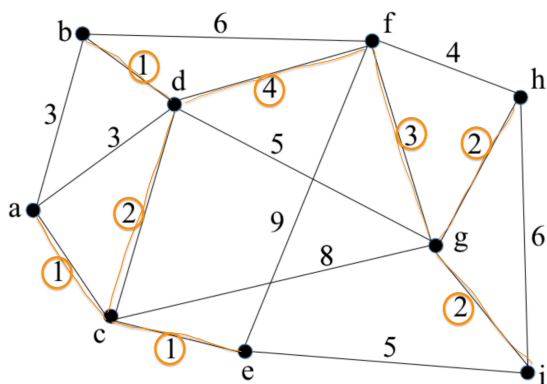
Prim 思想

非常像 Dijkstra。

每次：

| 从当前树向外扩展最小边。

最快判断 MST 用 **Kruskal**：按边权从小到大大选边；如果选了会形成环，就跳过；直到选够 $n-1$ 条边。



L-M H-L G-H G-K D-H D-E B-E A-B A-F I-K C-D I-J

total cost of an MST of the graph = 16

Prim 过程

1. 随便选起点
2. 看所有跨 cut 的边
3. 选最小
4. 加入树
5. 重复

时间复杂度

稀疏图（边少）

用 heap / priority queue, 智能剪枝： $O(m \log n)$

```
1 MST(G, s): // Parameters: graph G and start node s
2   mst = a tree with root node s
3   pq = an empty min-priority queue
4   For each edge e where e.from == s or e.to == s:
5     add e with priority e.w to pq
6   While pq is not empty:
7     remove the minimum priority edge e from pq
8     If e.from or e.to is not in mst:
9       u = the node among e.from or e.to that is not in mst
10      add e and u to mst
11      For each edge f where f.from == u or f.to == u:
12        If f.from or f.to is not in mst:
13          add f with priority f.w to pq
```

$O(n)$ rounds
can be amortized to $O(1)$
 $O(m)$ rounds
 $O(\log(m))$
overall true $O(n)$ times

overall $O(m)$ rounds
overall true $O(m)$ times
 $O(\log(m))$

dense graph（稠密图 $m \approx n^2$ ）

heap 优势消失，适合暴力扫描，用数组更快： $O(n^2)$

PQ 实现	Prim 复杂度
unsorted array	$O(n^2)$
sorted array	$O(mn)$
binary heap	$O(m \log n)$
Fibonacci heap	$O(m + n \log n)$

9. Articulation Point（割点）

定义

删除这个点后：图断开。

是“唯一连接器”：左边 ---- y ---- 右边

Tarjan 核心思想

DFS 时记录：

discovery time：第一次访问编号。

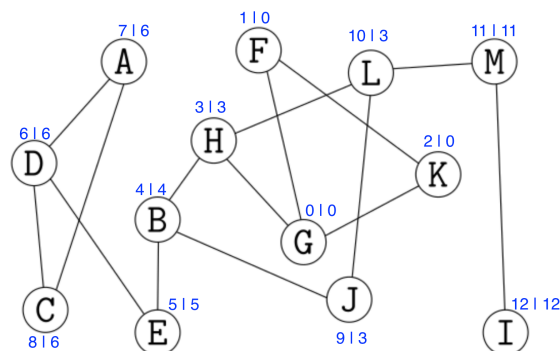
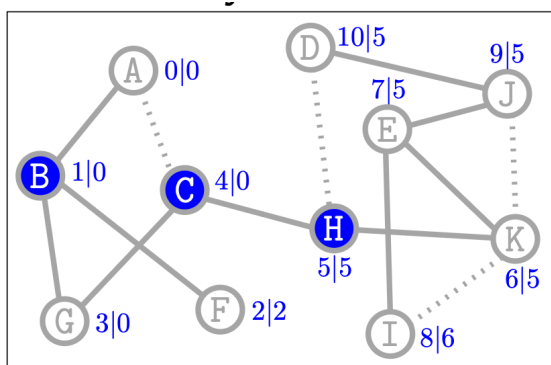
low-link value：能回到的最小编号。这棵树最远能逃回哪里。

low 的传播方向：

只能 child $\rightarrow$ parent 向上传。

不能 neighbor $\rightarrow$ neighbor 横向传播。

所以 $18|6$ ，不是 $8|5$



forward edge：真正“第一次发现新节点”的边

forward edges form a DFS tree。

判定条件

当 DFS 从 y 回退到 x ：

如果：

$$disc[x] \leq low[y]$$

则没有后门逃回祖先： x 是 articulation point。

10. Bridge（桥）

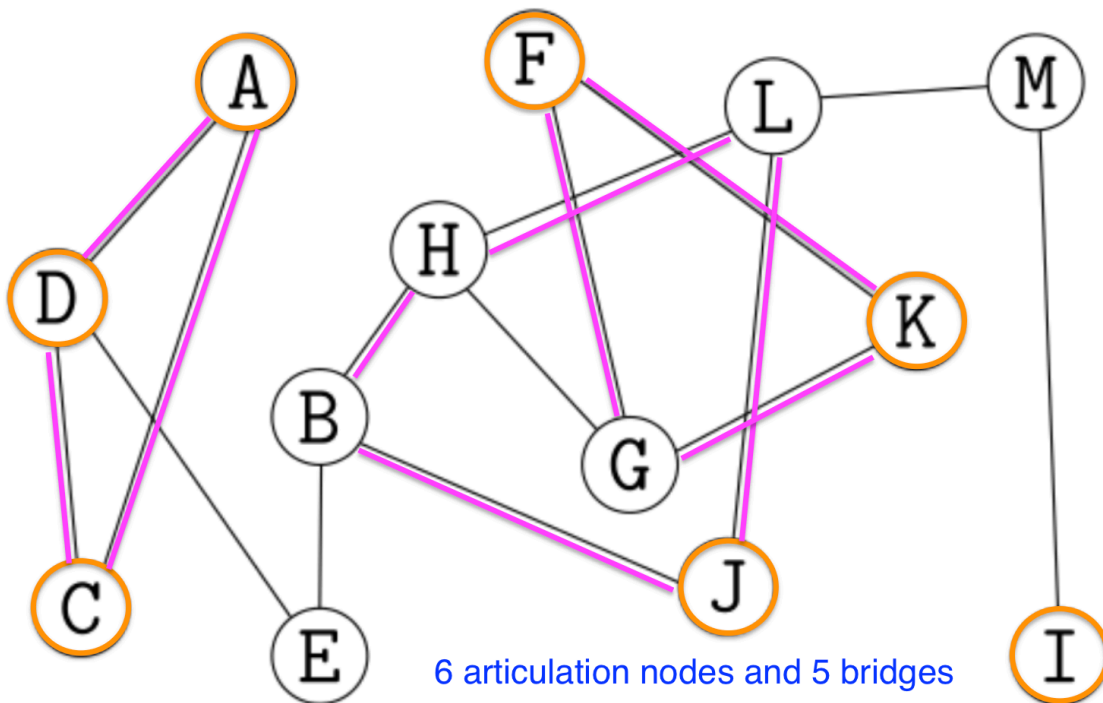
定义

删掉后图断开。

$x - y$ 是唯一连接方式。

判定条件

$$disc[x] < low[y]$$



tree 有个性质：

所有边都是 bridge (桥)
所有非叶子节点都是 articulation point (割点)

Round 2

1. 有向图 (Directed Graph / digraph)

无向图其实可以变成有向图：

如果原来有边： $A - B$

那么变成： $A \rightarrow B, B \rightarrow A$

也就是复制两条方向相反的边。

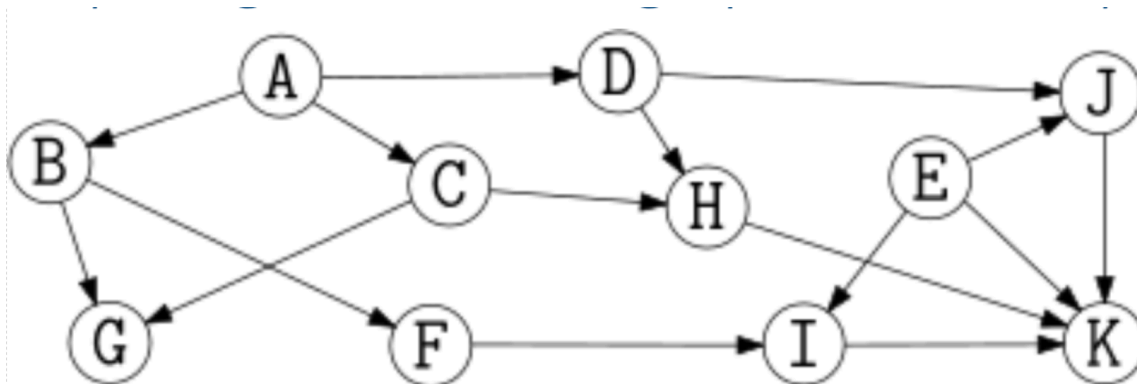
2. Topological Sort (拓扑排序)

核心概念

找一个节点顺序，使所有边都“向前”。

比如： $A \rightarrow C$

那么 A 必须出现在 C 前面。



legal topological orderings : A E C B G D J H F I K

illegal topological orderings : E A C B H D F G J I K

illegal topological orderings : A C B G D J H F E I K

应用

最经典：课程先修关系。

例如：Data Structure → Algorithm

那必须先学 DS 再学 Algo。

最重要结论

- 只要图里**没有环**，就**一定能**排出拓扑序；
- 反过来，如果图里有**环**，就**不可能**排出拓扑序；
- 而“没有环的有向图”换个名字就叫 **DAG**。

一个图：有拓扑排序 ⇔ 没有环 (cycle) ⇔ 是 DAG (Directed Acyclic Graph)

算法思想 (Kahn Algorithm)

不断删除 source node。

```

1 Topological-RSN(G): // Parameter: directed graph G
2   ord = an empty list // Will list the nodes in topological order
3   src = list of all source nodes
4   While src is not empty:
5       remove a node u from src and add it to ord
6       For each edge e with e.from == u:
7           remove e from the graph
8           If e.to became a source node:
9               add e.to to src
10  return ord

```

$O(n + m)$
 $O(n)$ rounds
 $O(1)$
overall $O(m)$ rounds
 $O(1)$
 $O(1)$

Source Node (源点)

没有 incoming edge 的点。

即：没人指向它。

步骤：

1. 找所有 source
2. 删除其中一个 (任选)
Source node (入度为 0 的点) 不一定只有一个，所以不同的选择会得到不同的拓扑序。
3. 删除它发出的边
4. 可能产生新的 source
5. 重复

为什么正确？

因为：source 前面没有依赖。

所以它一定可以最先出现。

时间复杂度

$O(n + m)$

- n = 点数
- m = 边数

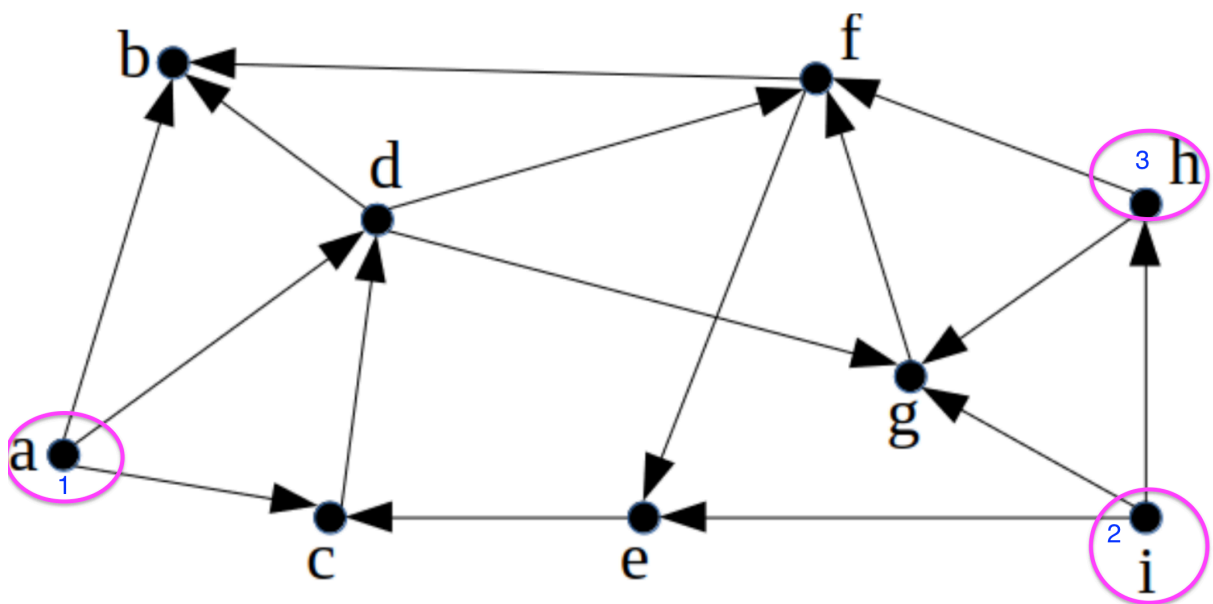
3. Cycle Detection (检测环)

如果：

- 图还没删空
- 但已经没有 source node

说明：

一定有环。



因为：每个点都有 incoming edge。

你一直往前追 predecessor : $u \leftarrow v \leftarrow w \leftarrow \dots$

由于点有限：一定会回到之前某个点。

于是形成 cycle。

4. Dijkstra Algorithm

单源最短路 shortest path tree (SPT)

求：从 s 到所有点的最短距离。

```
1 Dijkstra(G, s): // Parameters: directed graph G and start node s
2   spt = a tree with root node s
3   d(s) = 0 and d(u) = ∞ for u ≠ s           undefined distance = ∞ (infinity)
4   pq = an empty min-priority queue
5   For each edge e with e.from == s:         O(n) rounds
6     d(e.to) = e.w
7     add e with key e.to and priority d(e.to) to pq   can be amortized to O(1)
8   While pq is not empty:                     O(n) rounds
9     remove the minimum priority e from pq         O(log(n))
10    If e.to is not in spt:                      overall true O(n) times
11      add e and e.to to spt
12      For each edge f where f.from == e.to:      overall O(m) rounds
13        If f.to is not in spt:                  overall true O(m) times
14          d(f.to) = min{d(f.to), d(f.from) + e.w}
15          add f with key f.to and priority d(f.to) to pq   O(log(n))
```

核心贪心思想

每次：选择当前距离最小的未确定节点。

关键性质

一旦一个点被“确定”：它的最短距离永远不会再变。

这是 Dijkstra 正确性的核心。

为什么要求边权非负？

Dijkstra 假设：“已经最短”的点以后不会变小。

但负边会破坏这个性质。

复杂度

使用 heap : $O(m \log n)$

使用 unsorted array : $O(n^2)$

5. Bellman-Ford

能处理负边的最短路算法

```

1 Bellman-Ford(G, s): // Parameters: directed graph G and start node s
2   d(s) = 0 and d(u) = ∞ for u ≠ s
3   improved = list with start node s
4   For i = 1 to n-1:
5     nextImproved = an empty list
6     For each node u in improved:
7       For each edge e where e.from == u:
8         d(e.to) = min{d(e.to), d(e.from) + e.w}
9         If d(e.to) was improved:
10          add e.to to nextImproved
11   improved = nextImproved
12   If improved is empty:
13     step out of the main for-loop

```

$O(n)$
 $O(n)$ rounds
 $O(n)$ rounds
 overall $O(m)$ rounds
 $O(1)$

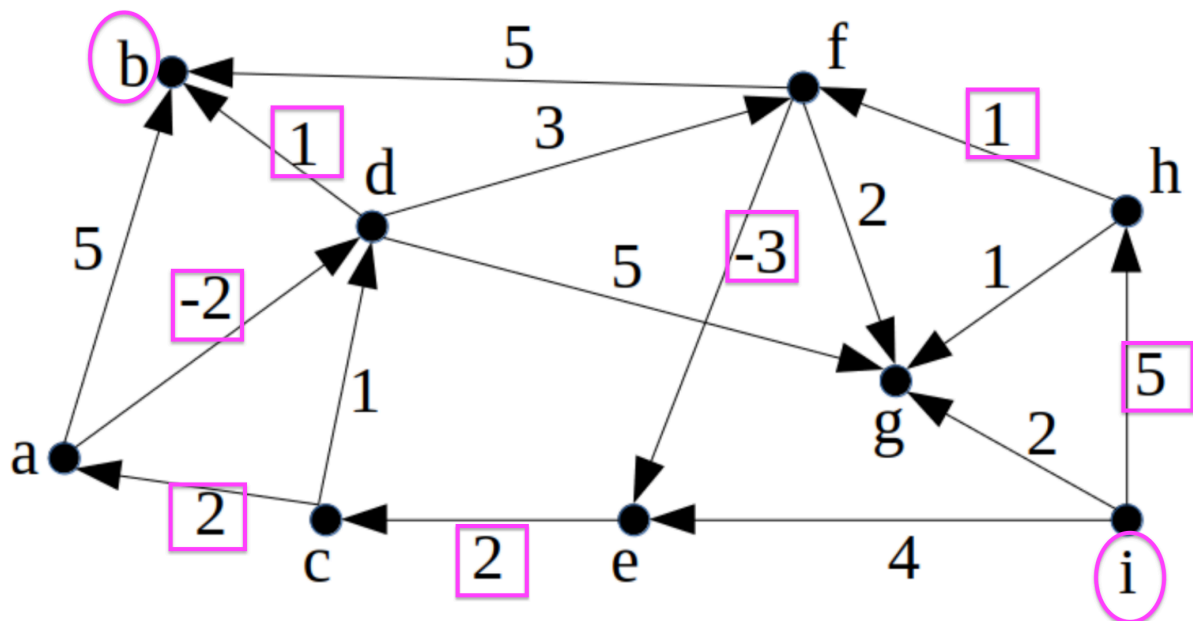
核心理想

暴力传播距离信息

对于每个点 u ，看所有的 incoming edge $v \rightarrow u$ 。

反复“松弛 (relax)”边：

如果： $d(u) > d(v) + w(v, u)$ ，就更新： $d(u) = d(v) + w(v, u)$



为什么有效？

因为：最短路的信息会一层层传播（异步传播模型）。

若最短路： $s \rightarrow v_1 \rightarrow v_2 \rightarrow \dots \rightarrow v_k$

那么：

- 第 1 轮后 v_1 正确
- 第 2 轮后 v_2 正确
- ...
- 第 k 轮后 v_k 正确

因此：最多做 $n-1$ 轮即可。

Bellman-Ford 的传播轮数

Bellman-Ford 的本质是：最短路径信息沿着边不断传播。更新公式为：

$$d(v) = \min(d(v), d(u) + w(u, v))$$

在 asynchronous update（异步更新）版本中，本轮刚更新出的距离值可以立刻继续使用，因此节点处理顺序会直接影响传播速度。

如果节点顺序与最短路传播方向一致，例如路径为： $s \rightarrow a \rightarrow b \rightarrow c$

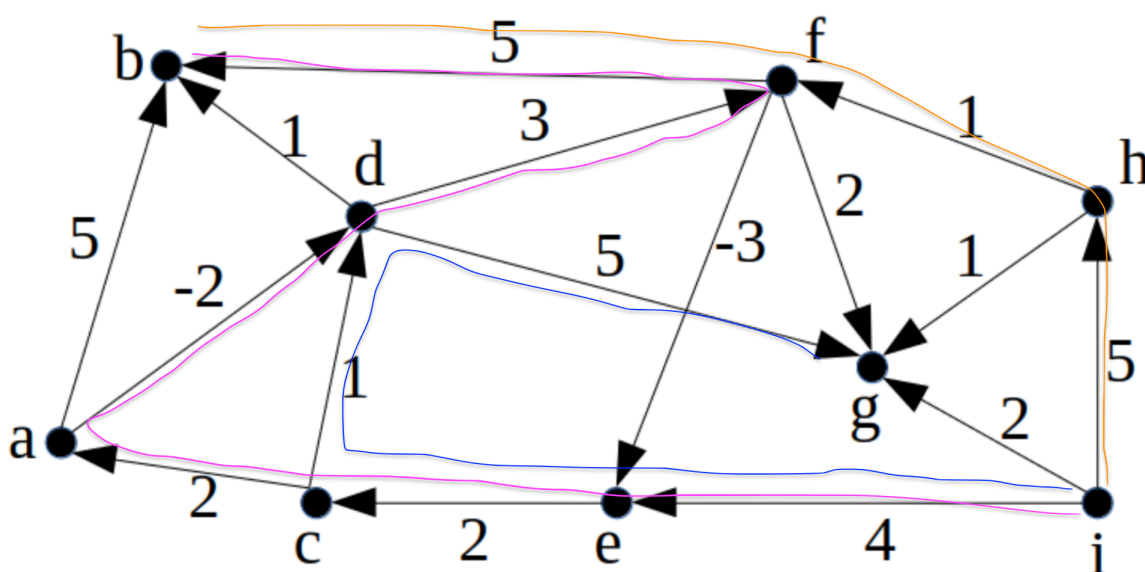
而节点处理顺序是： s, a, b, c

那么处理 s 时更新 a ，随后同一轮处理 a 时又能立刻更新 b ，接着处理 b 时再更新 c 。这样信息能够在同一轮连续传播，因此称为“顺着排，一轮走到底”。

如果整个图都存在这样一个固定顺序，使得所有有效的 shortest-path relax 都能在同一轮连续传播，那么只需要：

1轮完成所有更新
+1轮确认无更新

因此 smallest possible number of iterations 就是：2



相反，如果节点顺序与传播方向相反，例如路径仍然是： $s \rightarrow a \rightarrow b \rightarrow c$

但节点顺序变成： c, b, a, s

那么本轮直到最后处理 s 时， a 才被更新；但由于 a 本轮已经处理过了，所以只能等下一轮再更新 b 。接着再下一轮更新 c 。于是每一轮只能向前传播一步，这就是“反着排，一轮走一步”。

因此，如果某条最短路最长有： k edges

那么最慢需要： $(k+1)$ iterations

其中前 k 轮用于传播信息，最后 1 轮用于确认“已经没有任何更新”。

所以，smallest possible 并不一定总是 2。只有当存在一个固定节点顺序，使所有有效 relax 都能在同一轮连续传播时，答案才是 2；否则就需要更多轮。

时间复杂度

$O(n \cdot m)$

- n = 节点数 (number of vertices / nodes)
- m = 边数 (number of edges)

理论 worst-case :

Bellman-Ford 通常远慢于 Dijkstra。

实际运行 :

如果 图小, Bellman-Ford 不一定慢很多。

- 提前停止很快发生
- 最短路传播层数少

Bellman-Ford 的优势不是速度，而是：**支持负边与负环检测。**

6. Negative Cycle (负环)

什么是负环？

一个 cycle：边权和 < 0

为什么危险？

因为 你可以无限绕圈：

距离会趋向： $-\infty$

于是：“最短路”根本不存在。

比如某段运输/行动：

走这段路会得到补贴、返券、返现（等价于“成本是负的”）

如果这些负成本边能组成一个环：绕一圈还能净赚

那你就可以无限绕圈“刷补贴”

If a shortest path contains a cycle, the cycle either:

is redundant (overall cost is 0; can be removed without affecting the cost), or
the cycle has overall negative cost (is a “negative cycle”).

“允许负环，但禁止重复边”会变成非常困难的问题。所以大家统一禁止负环。

如何检测？

Bellman-Ford 做： n 轮。

如果：第 n 轮还能更新距离 $\Rightarrow$ 有负环。

负环在实际中通常不是“真的存在无限负成本”，而是一个强烈信号：模型里有套利机会、规则漏洞、数据错误或约束矛盾。

所以算法上要检测负环（Bellman-Ford 第 n 轮还能松弛），现实上要去修系统/修数据/加限制。

7. Floyd-Warshall

All-pairs shortest paths 全点对最短路

求：所有点之间的最短路。

```
1 Floyd-Warshall(G): // Parameter: directed graph G
2   For i = 1 to n:
3     For j = 1 to n:
4       If edge e from v[i] to v[j] exists:
5         D[i][j] = e.w
6         spEdge[i][j] = e
7       Else:
8         D[i][j] = ∞
9         spEdge[i][j] = null
10  D[i][i] = 0
11  For k = 1 to n:
12    For i = 1 to n:
13      For j = 1 to n:
14        If D[i][j] > D[i][k] + D[k][j]:
15          D[i][j] = D[i][k] + D[k][j]
16          spEdge[i][j] = spEdge[i][k] → first edge on path from v[i] to v[k]
```

核心 Dynamic Programming 思想

定义： $D[i][j]$

表示： i 到 j 的最短距离。

最关键递推

如果经过 k 更短：

则：

$$D[i][j] = \min(D[i][j], D[i][k] + D[k][j])$$

含义

原来： $i \rightarrow j$

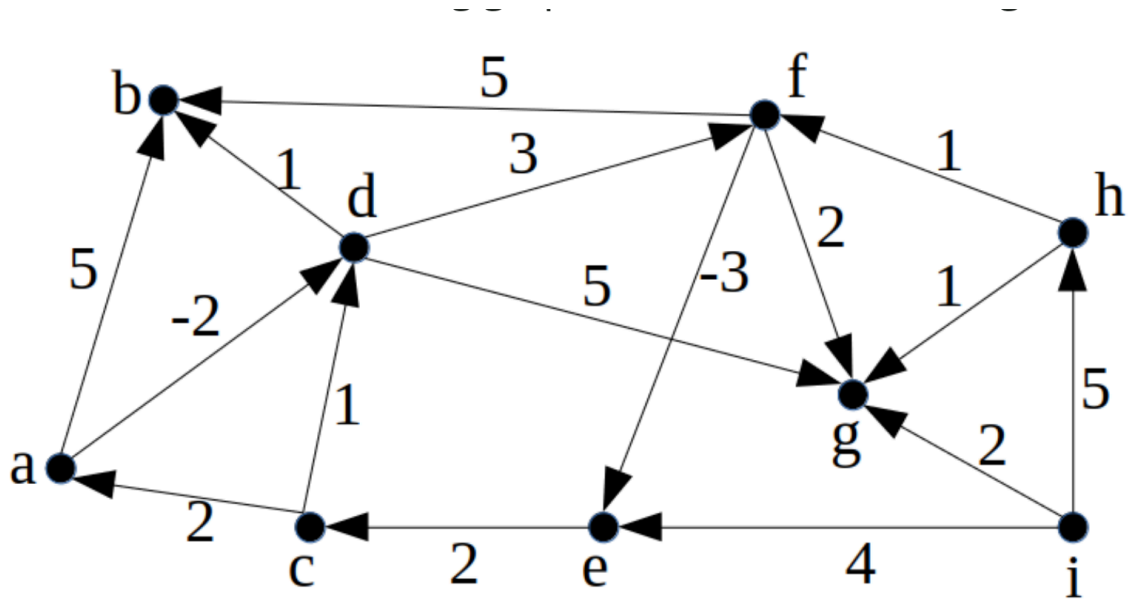
现在尝试： $i \rightarrow k \rightarrow j$

看看是否更短。

D表：从 i 到 j 的最短距离，所有 pair 的最短距离

如果最终 $D[i][i] < 0$ ，则图存在负环

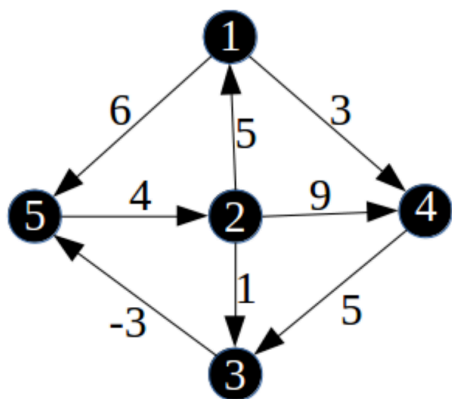
SpEdge表：shortest path edge table，从 i 到 j 的最短路第一步怎么走，如何恢复具体路径



a,b,c,d,e,f,g,h,i

iteration 1: a d f e

iteration 2: a d f e c



	1	2
1	0	∞
2	5	0
3	∞	∞
4	∞	∞
5	∞	4

iteration k=1:

	1	2	3
1	0	∞	∞
2	5	0	1
3	∞	∞	0
4	∞	∞	5
5	∞	4	∞

iteration k=2:

	1	2	3
--	---	---	---

1	0	∞	∞
2	5	0	1
3	∞	∞	0
4	∞	∞	5
5	$9(5 \rightarrow 2)$	4	$5(5 \rightarrow 2)$

iteration k=3:

	1	2	3
1	0	∞	∞
2	5	0	1
3	∞	∞	0
4	$7(4 \rightarrow 3)$	$2(4 \rightarrow 3)$	5
5	$9(5 \rightarrow 2)$	4	$5(5 \rightarrow 2)$

iteration k=4:

	1	2	3
1	0	$7(1 \rightarrow 4)$	$8(1 \rightarrow 4)$
2	5	0	1
3	∞	∞	0
4	$7(4 \rightarrow 3)$	$2(4 \rightarrow 3)$	5
5	$9(5 \rightarrow 2)$	4	$5(5 \rightarrow 2)$

iteration k=5:

	1	2	3
1	0	$7(1 \rightarrow 4)$	$8(1 \rightarrow 4)$
2	5	0	1
3	$6(3 \rightarrow 5)$	$1(3 \rightarrow 5)$	0
4	$7(4 \rightarrow 3)$	$2(4 \rightarrow 3)$	5
5	$9(5 \rightarrow 2)$	4	$5(5 \rightarrow 2)$

三层循环

```
for k
  for i
    for j
```

枚举：

- 中转点 k
- 起点 i
- 终点 j

时间复杂度

$O(n^3)$

总结

算法	用途	能否负边	能否负环	复杂度	核心思想
Topological Sort	DAG 排序	无关	不允许环	$O(n+m)$	不断删 source
Dijkstra	单源最短路	✗	✗	$O(m \log n)$	每次确定当前最近点
Bellman-Ford	单源最短路	✓	可检测	$O(n \cdot m)$	不断用已有最优值更新更优解
Floyd-Warshall	全点最短路	✓	可检测	$O(n^3)$	不断用已有最优值更新更优解

Round 3

Network Flow (网络流)

一个网络中，最多能从 source 运送多少东西到 sink？

- 水管输水
- 公路运输
- 网络传输
- 电流流动

1. Flow Network (流网络)

网络流本质上是：带容量的有向图 A directed weighted graph

- Edge weight 边权 = flow capacity of an edge (容量)
- source = 起点
- sink = 终点

例如：A → B : capacity 8

意思 最多只能运：8 单位流量。

2. 什么是 Flow？

每条边有：flow / capacity

例如 5 / 8 表示：

- 当前流量 = 5
- 最大容量 = 8

并且必须满足： $e.f < e.w$

即：流量不能超过容量

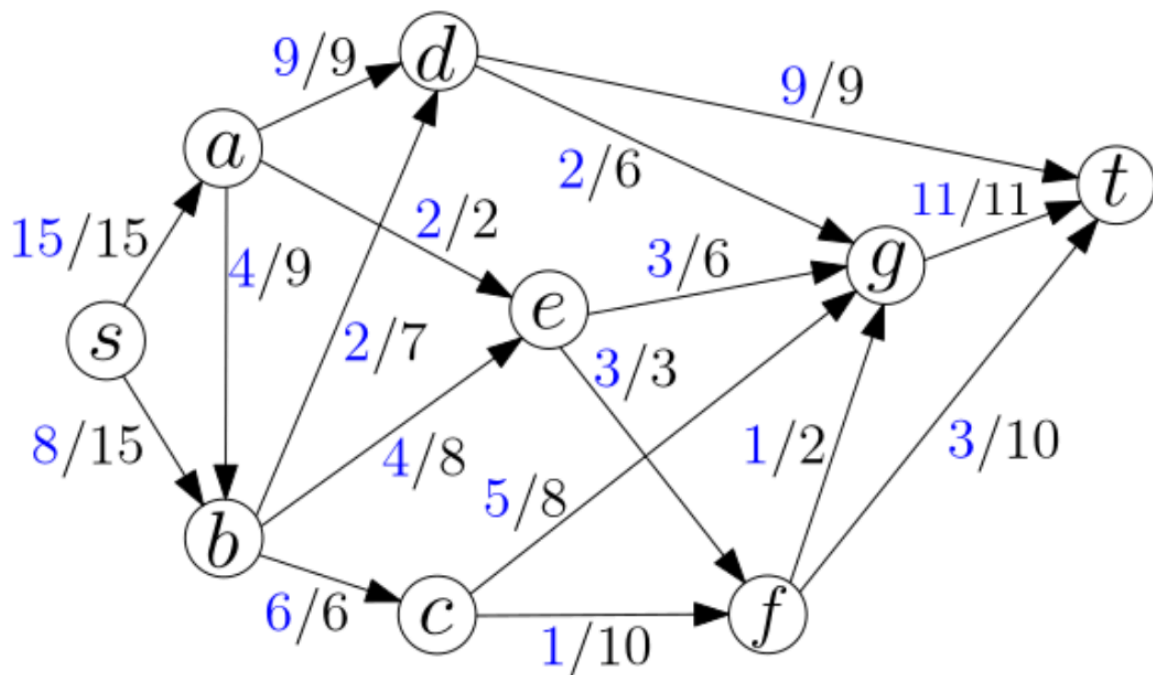
3. Flow Conservation (流量守恒)

对于除了 source 和 sink 之外的每个点：流入 = 流出

即：What comes in must go out

水进多少，就必须流出去多少。

快速判断 network flow legality：看“节点（不包括source/sink）有没有凭空制造/消失流量”



4. Maximum Flow (最大流)

从 source 到 sink 最大能送多少流量？

“快递包裹传送链”：整个系统一天最多送多少（完整经过整条路径）

5. Minimum Cut (最小割)

把图切成两边：

- source 在左
- sink 在右

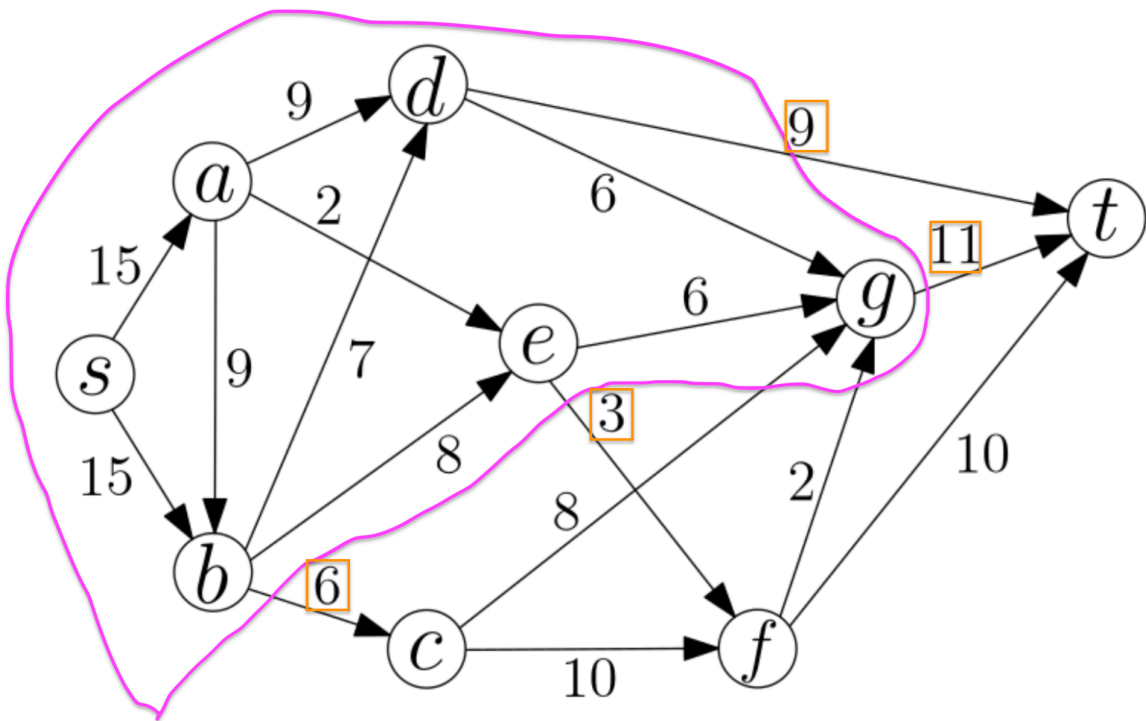
所有跨过去的边 叫：cut edges

这些边容量总和 叫：cut capacity

最小的那个 叫：minimum cut

最经典定理：Max Flow = Min Cut

最大能送多少流量 = 最小瓶颈切口的容量。



6. Augmenting Path (增广路)

思想：不断寻找还能增加流量的路径

例如：source $\rightarrow$... $\rightarrow$ sink

如果路径上每条边还有剩余容量，就还能加流。

两种边

(1) Forward Edge

正常方向还有空位： $e.f < e.w$

可以继续加流。

(2) Reverse Edge

反向边表示：可以撤销之前的流

意思：之前可能走错路了，现在允许把原来的流“退回来”重新安排。

本质：**flow rerouting (流量重定向)**

一条 augmenting path 能增加多少，不是看某一条边，而是看整条路径里最弱的那条边。

7. Bottleneck Edge (瓶颈边)

一条 augmenting path 能增加多少流量，

取决于：剩余容量最小的那条边，即：**bottleneck**

例如 路径剩余容量：5, 3, 7, 2, 8

那么最多只能增加：2

因为最小那条会先满。

8. Ford-Fulkerson (基础最大流)

```
1 MaxFlow(G, s, t): // Parameters: graph G, source s and sink t
2   While a new augmenting path is found:
3     add bottleneck flow along the augmenting path
```

$O(F)$ rounds
 $O(m)$

Step 1

初始化 flow = 0

Step 2

不断寻找 augmenting path

Step 3

沿路径增加 bottleneck flow

Step 4

找不到 augmenting path 时停止

此时得到 maximum flow。

9. 时间复杂度

如果容量是整数：每次 augment 至少增加 1

- 最大流 = F
- 边数 = m

则： $O(mF)$

问题：如果 F 很大 就会很慢。

10. Shortest Augmenting Path (Edmonds-Karp)

改进：每次找最短 augmenting path, 用：BFS

于是引入：Level Graph (层次图)

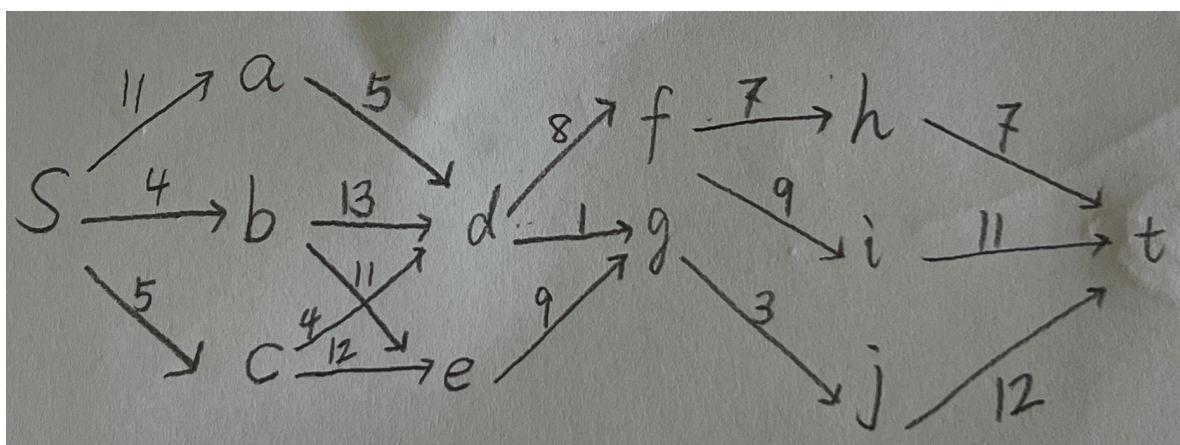
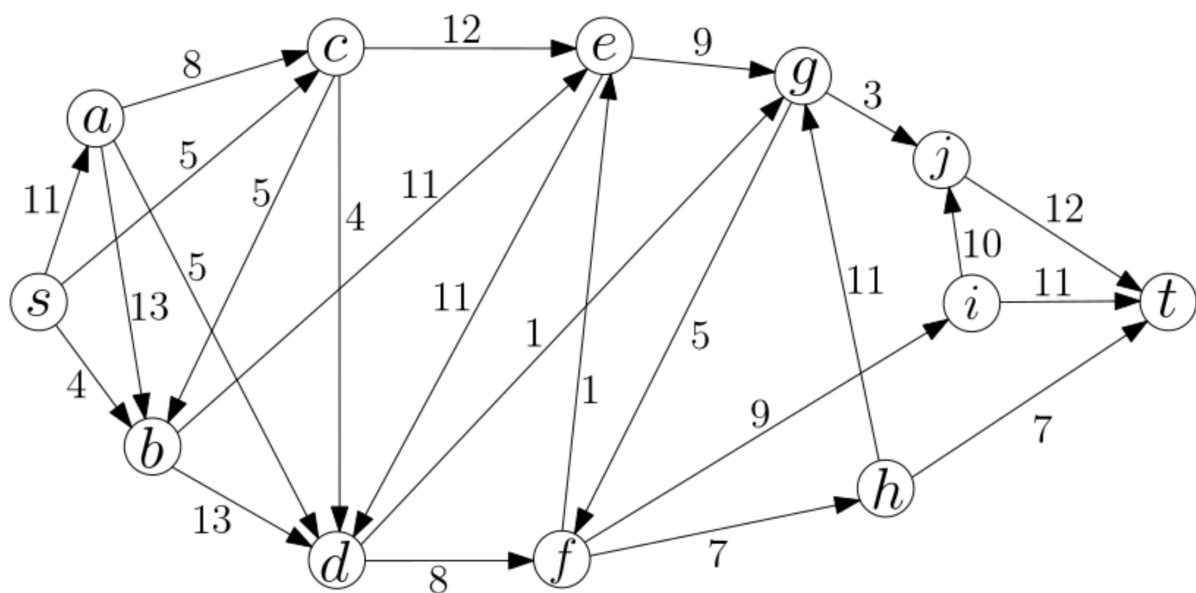
只保留：从第 i 层 $\rightarrow$ 第 $i+1$ 层的边

其它边 全部不要: 包括 同层边, 往回走的边, 跨两层的边

节点按：到 source 的边数距离 分层。

性质：新的 shortest augmenting path 不会比之前更短

因此复杂度改善为： $O(m^2 n)$



Maximal augmenting path (multiple path)

1. $s \rightarrow a \rightarrow d \rightarrow f \rightarrow h \rightarrow t$
bottleneck = $\min(11, 5, 8, 7, 7) = 5$

推 5 后, $a \rightarrow d$ 满了, $f \rightarrow h, h \rightarrow t$ 原来 7, 已经用了 5, 只剩 2。

2. $s \rightarrow c \rightarrow d \rightarrow f \rightarrow h \rightarrow t$
bottleneck = $\min(5, 4, 3, 2, 2) = 2$

推 2 后, $f \rightarrow h, h \rightarrow t$ 满了。

3. $s \rightarrow c \rightarrow d \rightarrow f \rightarrow i \rightarrow t$
bottleneck = $\min(3, 2, 1, 9, 11) = 1$

推 1 后, $d \rightarrow f$ 满了。

```
4. s → c → d → g → j → t
   bottleneck = min(2,1,1,3,12) = 1
```

推 1 后, `c→d, d→g` 满了。

```
5. s → b → e → g → j → t
   bottleneck = min(4,11,9,2,11) = 2
```

现在 `g→j` 满了。

总共增加： $5 + 2 + 1 + 1 + 2 = 11$

11. Dinitz Algorithm

Edmonds-Karp：一次 BFS, 只增广一条路径。

送一点点 → 重新 BFS → 再送一点点

Dinitz：一次 level graph 内做很多 DFS, 尽可能一次性塞满。

同一层全部送完再 BFS

流程：

BFS

建立 level graph

DFS

不断找 blocking flow, 直到这层图无法继续增广。

复杂度： $O(m n^2)$

12. Bipartite Matching (二分图匹配)

这是最大流最经典应用。

问题：两组人 L 和 R 有兼容关系。

问：最多能配多少对（左边每个点最多选一次，右边每个点也最多选一次）？

建图方法：

source → **L**

容量 1

L → **R**

如果 compatible

容量 1

R → **sink**

容量 1

跑 maximum flow。

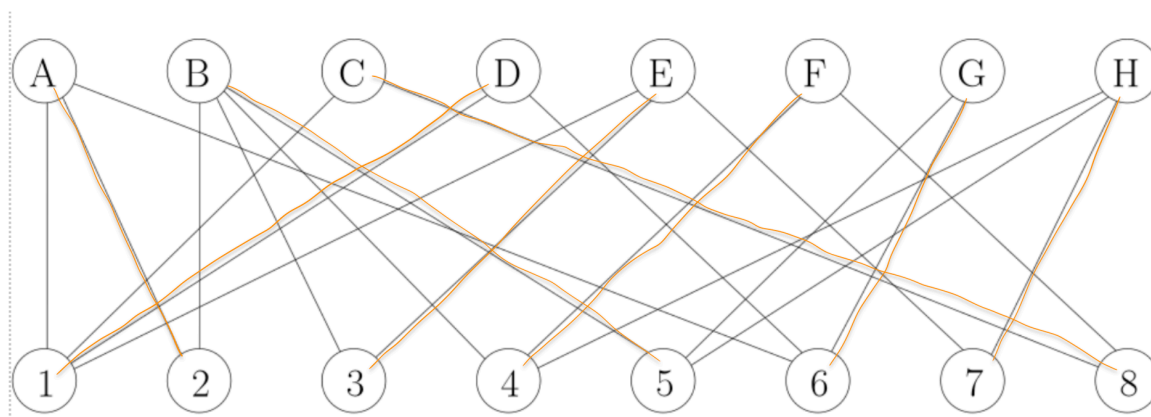
因为：容量都是 1

所以：每个人最多匹配一次

最大流 = 最大匹配数。

不是单纯“谁先抢到算谁的”，而是允许已经匹配的人重新换位置，给更多人腾出空间。

- 课程选导师（这个学生 愿意跟 这个导师），最多能安排多少学生？
- 工作招聘（这个员工 能胜任 这个岗位），最多能安排多少人上岗？
- 芯片设计（这个核心 能运行 这个任务），最多能同时安排多少任务？



13. Edge-disjoint Paths（边不相交路径）

问题：最多多少条路径互不共享边？

方法：

把每条边容量设为：1

因为：一条边最多只能被一个 flow 使用。

于是：最大流 = 最大 edge-disjoint paths 数

复杂度总结

“不断重新分配资源，使 source → sink 总流量最大化”

- augmenting path = 找新路线
- reverse edge = 撤销旧路线
- bottleneck = 当前瓶颈
- min cut = 整个网络的最终瓶颈

在大多数图里：边数 m 通常远大于点数 n

算法	思想	复杂度
Ford-Fulkerson	任意 augmenting path	$O(mF)$
Edmonds-Karp	BFS shortest augmenting path	$O(m^2n)$

算法	思想	复杂度
Dinitz	level graph + DFS blocking flow	$O(mn^2)$